PROJECT REPORT

FOR

Foundations of Artificial Intelligence (CS23533)

Rajalakshmi Engineering College

Department of Computer Science and Engineering

SEARCH & OPTIMIZATION PLAYGROUND

A PROJECT REPORT

Submitted by

240701610 - Yashwanth S

240701596 - Vishaali S



November, 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "Search & Optimization Playground" is the bonafide work of **Yashwanth S (Register No. 240701610)** and **Vishaali S (Register No. 240701596)**, who carried out the project work under the supervision of an Assistant Professor, Department of Computer Science and Engineering, Rajalakshmi Engineering College.

SIGNATURE OF THE FACULTY IN-CHARGE

Submitted for the Practical Examination held on: _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**4**
1	**INTRODUCTION**	**5**
1.1	Background	5
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Scope	5
2	**LITERATURE REVIEW**	**6**
2.1	Graph Search and Heuristics	6
2.2	Local Search and Optimization	6
2.3	Adversarial Game Search	6
2.4	Constraint Satisfaction	7
2.5	Project Context	7
3	**PROPOSED SYSTEM**	**8**
3.1	Architecture Diagram	8
3.2	User Interface and Workflow	9
3.3	Data Flow and Storage	9
4	**MODULES DESCRIPTION**	**10**
4.1	Pathfinding Lab	10
4.2	N-Queens Optimization Lab	10
4.3	Game AI Lab	10
4.4	Sudoku Lab	11
4.5	Timetabling Lab	11
4.6	Shared Playback and Dashboard	11
5	**IMPLEMENTATION AND RESULTS**	**12**
5.1	Experimental Setup	12
5.2	Results	12
6	**CONCLUSION AND FUTURE WORK**	**14**
6.1	Conclusion	14
6.2	Future Work	14
7	**REFERENCES**	**15**

ABSTRACT

This project presents a browser based educational workbench for studying search, optimization, adversarial game search, and constraint satisfaction in Foundations of Artificial Intelligence. The application combines five interactive learning areas: weighted grid pathfinding, N-Queens, Tic-Tac-Toe, Sudoku, and timetabling. Each run executes in a module Web Worker and records state snapshots, plain language explanations, pseudocode positions, and task-specific metrics. Learners can pause, step backward or forward, inspect frontiers and domains, and compare algorithms on copies of the same input. The implementation uses HTML, CSS, vanilla JavaScript, SVG, Web Workers, and browser LocalStorage. On the documented weighted grid, breadth-first search returns 13 moves at cost 93, while uniform-cost search and A* return 19 moves at cost 19. On the Sudoku teaching instance, forward checking reduces assignments from 81 to 41 and backtracks from 13 to 5. These results are tied to the supplied presets and settings. The project verification record reports 18 of 18 automated checks passed. The work makes algorithm decisions and trade-offs visible through an interactive interface without a server-side computation layer.

1 INTRODUCTION

1.1 Background

Search and constraint solving support route planning, resource allocation, puzzles, games, and scheduling. Yet a final answer hides how an algorithm chooses a state, rejects a candidate, or recovers from failure. This project makes those steps visible through recorded traces and visual problem states.

1.2 Problem Statement

Students often see search algorithms as pseudocode or static examples, though their differences lie in intermediate state: queue order, path cost, heuristics, domains, pruning, and backtracking. Fair comparisons also need identical inputs and clearly defined metrics. This project addresses these needs with editable problems, replay, and within-lab comparison.

1.3 Objectives

The project provides editable examples of graph search, local optimization, adversarial search, and constraint satisfaction. It records explained decisions, highlights pseudocode, reports algorithm-specific metrics, and compares methods on identical inputs. Shared playback lets learners revisit states without rerunning.

1.4 Scope

Five labs and a comparison dashboard run as a static browser application. Configurations and recent summaries are stored per browser origin. The grid is fixed-size, Sudoku returns its first valid solution, and each timetable session occupies one slot.

2 LITERATURE REVIEW

2.1 Graph Search and Heuristics

Dijkstra's shortest-path work established a systematic way to find minimum-cost routes in a graph with nonnegative edge costs (Dijkstra, 1959). Hart, Nilsson, and Raphael later gave a formal basis for incorporating problem-specific heuristic information into minimum-cost graph search and proved an optimality property for a class of heuristic strategies (Hart, Nilsson, and Raphael, 1968). These ideas inform the project's comparison of breadth-first search, uniform-cost search, Greedy best-first search, and A*. The visualization makes a useful distinction visible: breadth-first search prioritizes the number of moves, while uniform-cost search and A* prioritize accumulated path cost under the project's documented cost and heuristic assumptions.

2.2 Local Search and Optimization

Hill climbing improves a candidate by selecting a better neighboring state, but it may stop at a local minimum or plateau. Simulated annealing adds a controlled chance of accepting a worse move. Kirkpatrick, Gelatt, and Vecchi introduced simulated annealing as a method for combinatorial optimization, drawing an analogy to the cooling of physical systems (Kirkpatrick, Gelatt, and Vecchi, 1983). The N-Queens lab exposes both methods on the same conflict objective and records the temperature, proposal, acceptance decision, current score, and best score. The finite schedule is an experiment setting, so success rates describe only the tested seeds and budget.

2.3 Adversarial Game Search

Minimax selects a move by assuming that both players choose actions that improve their own outcomes. Alpha-beta pruning uses current lower and upper bounds to skip branches that cannot change the root decision. Knuth and Moore analyzed the correctness and search behavior of alpha-beta pruning (Knuth and

Moore, 1975). The Game AI lab uses a full-depth Tic-Tac-Toe tree and displays explored nodes, terminal values, cutoffs, and the bounds associated with pruned branches.

2.4 Constraint Satisfaction

A constraint satisfaction problem represents a task through variables, domains, and constraints. Backtracking assigns values and undoes choices after a conflict. Forward checking additionally removes inconsistent values from unassigned neighboring variables after each accepted assignment. Haralick and Elliott studied forward checking as a look-ahead procedure for improving tree-search efficiency in constraint problems (Haralick and Elliott, 1980). The project applies these ideas to Sudoku cells and timetable sessions, making domain reduction and restoration visible during replay.

2.5 Project Context

The project brings these algorithm families into one browser workbench. Its learning focus is the observable process: students can inspect the state of the frontier, game tree, or constraint domains, and then compare operation counts on the same input. It does not claim that one algorithm is best across different problem types; metrics are reported within each learning area and interpreted in the context of the chosen preset.

3 PROPOSED SYSTEM

The proposed system is a browser based application that separates problem configuration, algorithm execution, playback, and comparison. The user edits a problem through the interface. A copied configuration is sent to a module Web Worker, which invokes the selected algorithm engine. The engine returns recorded states, a final result, and cumulative metrics. The interface can replay those states or submit equivalent input copies to the comparison engine.

3.1 Architecture Diagram

Figure 3.1 shows the main data flow. Algorithm engines compute without touching the document object model. Recorded snapshots flow through the shared playback controller to SVG and HTML views. Final outputs and metrics flow to the comparison engine and dashboard. Browser LocalStorage retains saved configurations and result summaries.

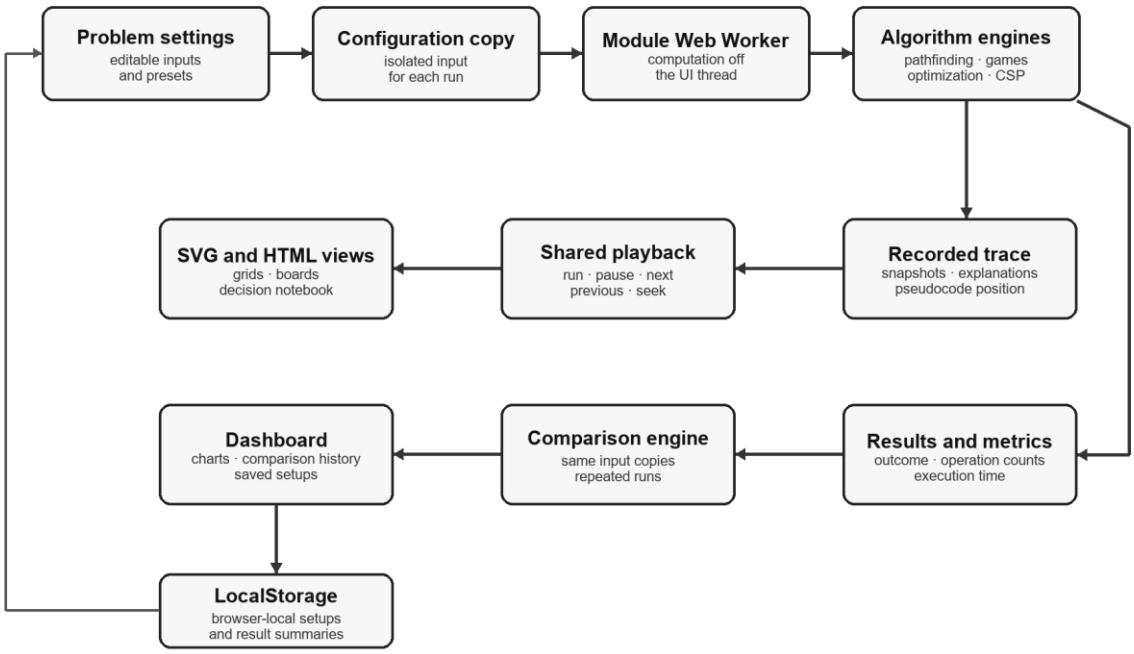


Figure 3.1. Architecture of the Search & Optimization Playground

3.2 User Interface and Workflow

The dashboard provides entry points to all five learning areas and a panel for recorded comparisons. Within a lab, the learner selects an algorithm and preset, edits the input if needed, and runs the experiment. Playback controls expose the recorded decisions; the decision notebook explains the selected state and highlights the corresponding pseudocode. Figure 3.2 shows the dashboard interface used to reach the labs and comparison history.

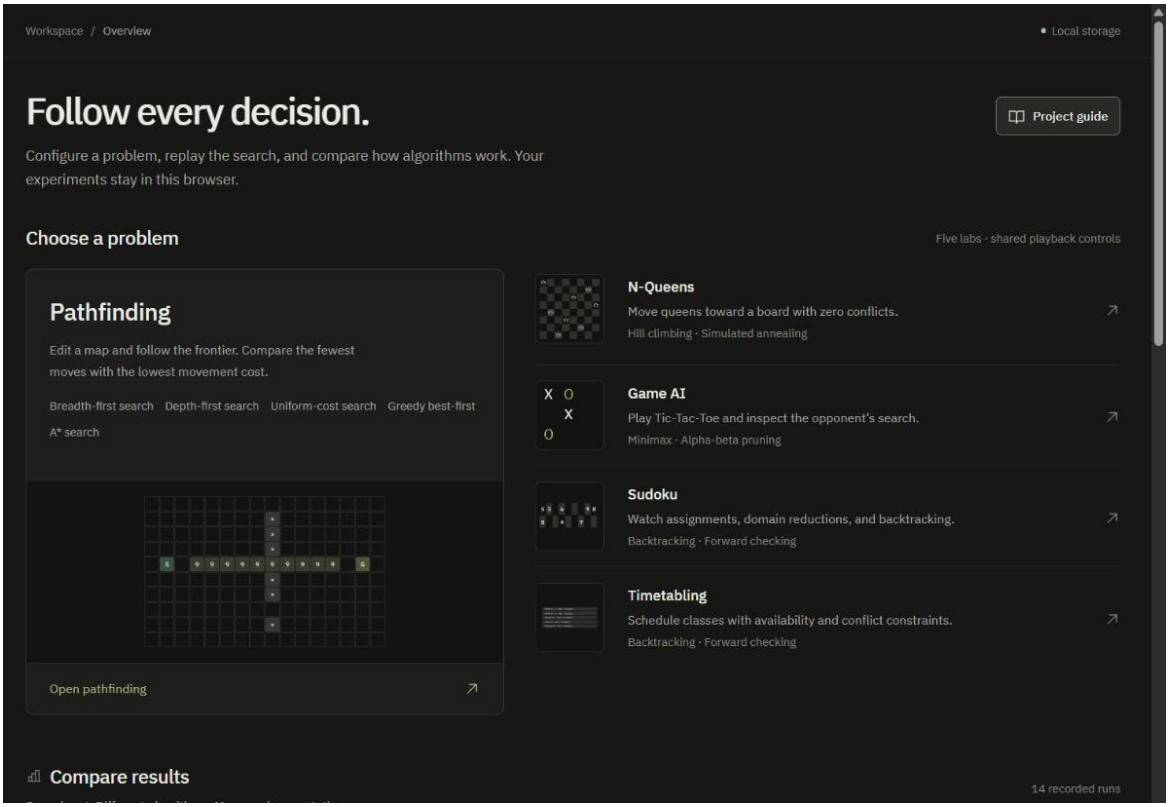


Figure 3.2. Dashboard interface with the learning areas and comparison panel

3.3 Data Flow and Storage

Each run sends a copy of the input to the worker. The trace stores events, state snapshots, metrics, pseudocode positions, and explanations. Playback displays saved snapshots without rerunning the algorithm. LocalStorage retains configurations and recent summaries; complete traces stay in memory during the active session.

4 MODULES DESCRIPTION

The application is organized around five problem-specific labs and one shared playback and dashboard layer. The labs use different state representations, but each follows the same workflow: configure an input, run an algorithm, inspect its decisions, and compare supported alternatives.

4.1 Pathfinding Lab

Pathfinding uses an editable 10 by 16 grid with a start, a goal, walls, and positive terrain entry costs. The lab implements breadth-first search (BFS), depth-first search (DFS), uniform-cost search (UCS), Greedy best-first search, and A*. The interface displays the active frontier, visited cells, parent links, path, and the g, h, and f values used by cost-based searches. BFS minimizes moves when all step costs are equal; UCS minimizes cumulative cost; A* uses a Manhattan-distance heuristic multiplied by the minimum traversable entry cost. DFS and Greedy do not guarantee the lowest-cost route.

4.2 N-Queens Optimization Lab

The N-Queens lab supports board sizes from 4 through 12. A board is represented by one row position per column, and the objective counts attacking queen pairs. Hill climbing evaluates the single-queen neighborhood and accepts a strictly improving arrangement; it stops when no move improves the score. Simulated annealing uses a seeded pseudo-random generator and can accept a worse move with probability based on the change in conflict score and current temperature. The user can inspect proposals, acceptance events, temperature, current score, best score, and the score chart.

4.3 Game AI Lab

The Game AI lab contains a playable Tic-Tac-Toe board with Minimax and optional alpha-beta pruning. The search evaluates terminal boards to depth nine and assigns utility from the AI's perspective: positive for a win, negative for a

loss, and zero for a draw. The learner can expand the retained search tree and inspect MAX and MIN choices, alpha and beta bounds, cutoffs, terminal evaluations, and the selected move. Move ordering can be natural or center-first; it can change the number of nodes visited and the choice among equally optimal moves.

4.4 Sudoku Lab

Sudoku is modeled as a standard 9 by 9 constraint problem. Each empty cell is a variable, its candidate digits form a domain, and cells sharing a row, column, or 3 by 3 box are peers. Backtracking checks candidates against assigned peers. Forward checking also removes conflicting values from unassigned peers and restores them after a failed branch. The lab shows candidates, assignments, rejection, pruning, restoration, and backtracking. Duplicate clues are rejected before search; exhaustive failure and a search limit are reported as different outcomes.

4.5 Timetabling Lab

The timetabling model treats each required session as a variable. A domain value pairs an allowed time slot with an allowed room. Teacher and room availability restrict initial domains; simultaneous sessions cannot share a teacher, group, or room. The backtracking and forward-checking engines apply the same constraint checks, while the latter removes conflicting slot and room choices from unscheduled sessions. The current teaching model assigns one slot to each session and does not include duration, room capacity, or soft preferences.

4.6 Shared Playback and Dashboard

Playback offers run, pause, step, reset, seek, and speed control; snapshots support reverse review without rerunning. The dashboard compares methods, charts metrics, saves setups, exports JSON, and uses shared inputs or paired seeds and boards.

5 IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The application uses HTML5, CSS3, vanilla JavaScript modules, SVG, module Web Workers, and browser LocalStorage. It has no frontend package installation or build step. To run it locally, start a static server from the project directory with python -m http.server 8000 --bind 127.0.0.1 and open http://127.0.0.1:8000 in a modern browser that supports ES modules, module workers, structuredClone, and LocalStorage. Node.js is optional for the project's automated checks and reproducible comparison-report script.

The documented comparison report was generated from unchanged default presets on 12 October 2026 with Node.js 26.4.0 on Windows x64. Deterministic comparisons use one warm-up and five timed runs. The N-Queens comparison uses 20 consecutive seeds and three timing repetitions per trial; success rates therefore describe that finite trial set. Computation timing excludes animation and rendering and varies by device and runtime. Operation counts and correctness outcomes are more useful for interpreting the examples than small timing differences.

5.2 Results

Table 5.1 summarizes selected recorded comparisons. The results are tied to the named presets and settings and are not universal performance guarantees.

Table 5.1. Selected outcomes from the documented preset comparisons

Learning area	Recorded comparison result
Weighted pathfinding	BFS: 13 moves, cost 93; UCS and A*: 19 moves, cost 19. Expansions: 113 (BFS), 142 (UCS), and 69 (A*).
N-Queens	Hill climbing stopped at a plateau with one conflict (0% success). Simulated annealing solved 30% of 20 seeded trials.
Tic-Tac-Toe	At depth 9, alpha-beta evaluated 18,297 nodes versus 549,946 for Minimax and recorded 4,237 cutoffs. The root utility matched under identical move ordering.

Learning area	Recorded comparison result
Sudoku	Forward checking used 41 assignments and 5 backtracks and removed 44 values; backtracking used 81 assignments and 13 backtracks.
Timetabling	Forward checking used 7 assignments and 2 backtracks and removed 13 values; backtracking used 21 assignments and 3 backtracks. All five sessions were assigned.

The weighted pathfinding case demonstrates that the route with the fewest moves need not have the lowest terrain cost. The game search comparison shows that alpha-beta pruning can reduce explored nodes while preserving the root utility. Forward checking reduced candidate attempts and backtracks in both the Sudoku and timetable examples, at the cost of performing domain filtering. The N-Queens result is stochastic and should be read with its trial count and finite budget.

The project verification record dated 13 October 2026 reports 18 automated checks passed, with no failures or skips, and browser exercises across all five labs. The browser integration record covers one browser; it does not report a cross-browser matrix or a full accessibility audit. The numerical comparison tables are reproduced from the project's generated report and raw configurations (YashwanthDevelops, 2026).

6 CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Search & Optimization Playground combines graph search, local optimization, game-tree search, and constraint satisfaction in a single browser based teaching tool. Its central contribution is an inspectable trace: learners can follow each decision, connect it to pseudocode, and compare measurable work on controlled inputs. The recorded examples show why algorithm choice depends on the objective and model. A path with fewer moves can cost more, alpha-beta pruning can reduce game-tree exploration, and forward checking can reduce assignments by pruning domains early. The project's browser-only architecture makes the labs easy to run while keeping each algorithm's state and limits visible.

6.2 Future Work

Future development can extend the fixed-size pathfinding grid, add dynamic variable ordering and stronger consistency procedures such as AC-3 for constraint problems, support Sudoku uniqueness checks and additional solutions, and model timetable duration, room capacity, and soft preferences. Further work can also add more games, improve trace export for long runs, and evaluate the interface across additional browsers and accessibility settings. These extensions build on the current separation between problem models, algorithm engines, playback, and comparison.

7 REFERENCES

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269-271. https://doi.org/10.1007/BF01386390

Haralick, R. M., and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence, 14(3), 263-313. https://doi.org/10.1016/0004-3702(80)90051-X

Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

Kirkpatrick, S., Gelatt, C. D., Jr., and Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671-680. https://doi.org/10.1126/science.220.4598.671

Knuth, D. E., and Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial Intelligence, 6(4), 293-326. https://doi.org/10.1016/0004-3702(75)90019-3

YashwanthDevelops. (2026). Search & Optimization Playground [Computer software and project documentation]. GitHub. https://github.com/YashwanthDevelops/Algo-Visualizer (accessed 20 October 2026).